

SauceDemo E-Commerce UI Automation Suite

Final QA Bootcamp Solo Project

Presented by: Ahmed

Role: QA Automation Engineer Candidate

SELENIUM WEBDRIVER • TESTNG • JAVA • APACHE MAVEN • PAGE OBJECT MODEL

Project Scope & Objectives

Core Project Objective

To design, develop, and execute an enterprise-grade automated UI testing suite for the SauceDemo e-commerce platform, ensuring functional stability across major user operations.

In-Scope Functional Areas

- End-to-End core business workflows (Happy Path).
- Form field validation and input constraint constraints.
- UI exception handling and negative scenario flows.
- Automated generation of native TestNG HTML execution reports.

Out-of-Scope Elements

- Backend structural performance testing (Load / Stress).
- Security scanning and penetration validation.
- Direct relational database constraints checks.
- Native Mobile Application testing (iOS / Android).

Framework Architecture & Tech Stack

The automation framework follows the industry-standard architectural best practices to maximize code reusability, structural maintainability, and clean decoupling.

Page Object Model (POM) Design Pattern

UI elements and locators are stored strictly inside dedicated Page classes (e.g., LoginPage, InventoryPage). Test execution classes exclusively manage state assertions, completely separating structural mechanics from testing business logic.

Apache Maven Build Tool

Manages dependencies and external framework library execution dynamically via the central pom.xml file config, eliminating local binary path variance dependencies.

Selenium WebDriver (v4.18.1)

Drives user interface behavior actions natively on Google Chrome. Implements Selenium 4 Manager for automated web driver binary life-cycle controls.

TestNG Testing Framework (v7.9.0)

Coordinates testing life-cycle triggers through structural assertions and configurations annotations (@BeforeMethod, @Test, @AfterMethod).

Automated Testing Workflows

Two complete end-to-end user flows have been mapped from empirical manual execution criteria and translated into robust automated regression scripts:

Workflow 1: The "Happy Path" E-Commerce Purchase Journey

Execution Pattern: Navigate to storefront → Secure authentication with valid profile (standard_user) → Select product catalog items ("Sauce Labs Backpack") → Update cart badge indicators → Fill multi-input shipping metadata forms → Finalize payment step transactions.

Core Validation Assertion: System must explicitly display confirmation header containing: "Thank you for your order!".

Workflow 2: Negative Validation & Application Isolation Constraints

Execution Pattern: Launch login state module → Submit isolated account credentials matching an administratively banned user data matrix (locked_out_user) → Trigger application submission controls.

Core Validation Assertion: Active intercept mechanisms must maintain visibility containment, ensuring the system returns validation element showing exactly: "Epic sadface: Sorry, this user has been locked out.".

Execution Metrics & TestNG Reporting

100%

Automation Pass Rate

All configured execution workflows finalized assertions successfully without a single functional regression drop.

Built-in Live Demo Adaptations

Implemented a unified utility helper delay sequence (**3-second thread pause intervals**) strategically distributed across interactive transactional steps.

Benefit: Allows evaluation board members to comfortably review automated cross-page navigation, data payload entry, and programmatic browser assertions in real time.

Native HTML Reporting Engine

Test runs compile full runtime data into structural emailable-report.html and index.html artifacts inside the test-output/project directories for stakeholders visual consumption.

Conclusion & Live Framework Demo

Thank you! Open for Questions & Technical Evaluation

Repository Link Synchronized & Available on GitHub